Práctica 1. Introducción a R

Laboratorio de Inteligencia Artificial

Facultad de Ciencias, UNAM

Nombre del alumno: Osorio Lopez Claudia Yarinet

Número de cuenta: 316256098

Fecha de publicación: 14 de septiembre 2026

Periodo de entrega: del 14 al 20 de septiembre 2026

Presentación

Durante las sesiones presenciales del curso el trabajo se realiza en Python. En este laboratorio vamos a usar **R**, un lenguaje pensado desde su origen para el análisis de datos y el cálculo estadístico. No se trata de aprender dos herramientas por gusto: buena parte de los métodos que veremos más adelante —clasificación bayesiana, árboles de decisión, agrupamiento— tienen en R una implementación directa y muy legible, y conviene tener ese segundo punto de vista.

Esta primera práctica tiene dos propósitos. El primero es que te familiarices con la sintaxis básica del lenguaje. El segundo es que uses esa sintaxis para trabajar con los contenidos del Tema I del curso, en particular la **representación de conocimiento lógico** y la idea de **agente**. R resulta especialmente cómodo para esto, porque los valores lógicos son un tipo de dato de primera clase y las operaciones sobre ellos se aplican de manera natural a colecciones enteras.

Cómo trabajar esta práctica

1. Abre este archivo en Google Colab.
2. Verifica que el entorno de ejecución sea **R**. Para ello entra al menú `Entorno de ejecución` → `Cambiar tipo de entorno de ejecución` y selecciona `R` en el campo de lenguaje. El archivo sigue siendo un `.ipynb`; lo único que cambia es el intérprete que ejecuta las celdas.
3. Lee las explicaciones y **ejecuta cada celda de código en orden**, con `Shift + Enter`. Muchas celdas dependen de variables definidas en celdas anteriores.
4. Resuelve los ejercicios de la sección final en las celdas vacías destinadas a ello.
5. Antes de entregar, usa `Entorno de ejecución` → `Reiniciar y ejecutar todo` para confirmar que el cuaderno corre de principio a fin sin errores.

Entrega

Entrega este mismo archivo `.ipynb` **con las salidas visibles**, a través de Google Classroom. Un cuaderno entregado sin haber sido ejecutado se considera incompleto.

Cualquier duda sobre la práctica debe plantearse por Classroom, de modo que la respuesta quede disponible para todo el grupo.

```
##%load_ext rpy2.ipython
```

---

1. Preparación del entorno

La primera celda no hace nada interesante, pero sirve como comprobante de que estás trabajando en el intérprete correcto. Si la salida menciona una versión de R, todo está en orden. Si en cambio obtienes un error de sintaxis, es señal de que el entorno sigue configurado en Python y debes cambiarlo antes de continuar.

```
R.version.string
```

```
'R version 4.6.1 (2026-06-24)'
```

Conviene también saber qué paquetes están disponibles sin necesidad de instalar nada. R trae de fábrica un conjunto de paquetes base que cubre de sobra lo que haremos hoy.

```
# search() muestra los paquetes cargados en la sesión actual.
search()
```

```
'.GlobalEnv' · 'jupyter:irkernel' · 'package:stats' · 'package:graphics' · 'package:grDevices' · 'package:utils' · 'package:datasets' · 'package:methods' · 'Autoloads' · 'package:base'
```

---

2. Lo mínimo indispensable de la sintaxis

Asignación y tipos

En R la asignación se escribe con la flecha `<-`. El signo `=` también funciona, pero por convención se reserva para pasar argumentos a las funciones, y seguir esa convención hace el código bastante más legible.

```
materia <- "Inteligencia Artificial"
creditos <- 10
es_optativa <- FALSE

# class() nos dice de qué tipo es cada valor.
class(materia)
class(creditos)
class(es_optativa)
```

```
'character'
'numeric'
'logical'
```

Nota que R imprimió los tres resultados. Dentro de un cuaderno, cualquier expresión que no se asigne a una variable se muestra en pantalla. Cuando quieras imprimir algo de forma explícita —por ejemplo dentro de un ciclo o una función— usa `print()`.

Vectores

El vector es la estructura fundamental del lenguaje. Se construye con `c()`, que viene de *combine*. Aquí es donde R empieza a separarse de lo que conoces de Python: prácticamente todas las operaciones trabajan sobre el vector completo, sin que haya que escribir un ciclo.

```
calificaciones <- c(8.5, 9.0, 6.7, 10.0, 7.2)

length(calificaciones)
mean(calificaciones)
max(calificaciones)
sort(calificaciones)
```

```
5
8.28
10
6.7 · 7.2 · 8.5 · 9 · 10
```

Indexación

Los índices en R **empiezan en 1**, no en 0. Este es el tropiezo más común para quien viene de otro lenguaje, así que vale la pena tenerlo presente desde ahora.

```
calificaciones[1]      # el primer elemento
calificaciones[c(2, 4)]  # el segundo y el cuarto
calificaciones[-1]     # todos MENOS el primero
```

```
8.5
9 · 10
9 · 6.7 · 10 · 7.2
```

Ese último caso merece atención: en R un índice negativo no cuenta desde el final, sino que **excluye** la posición indicada. Es una diferencia importante respecto a Python.

Operaciones vectorizadas

Cuando aplicas una operación a un vector, esta se aplica a cada elemento. El resultado es otro vector de la misma longitud.

```
calificaciones + 0.5
calificaciones * 2
round(calificaciones)
```

```
9 · 9.5 · 7.2 · 10.5 · 7.7
17 · 18 · 13.4 · 20 · 14.4
8 · 9 · 7 · 10 · 7
```

---

3. Valores lógicos

Aquí entramos ya en terreno del Tema I. En R los valores de verdad se escriben `TRUE` y `FALSE`, siempre en mayúsculas. Los operadores disponibles son los que esperarías:

Operador	Significado
`&`	conjunción
`|`	disyunción
`!`	negación
`xor()`	disyunción exclusiva
`==`	equivalencia

Lo interesante es que estos operadores también son vectorizados, lo cual será la clave de toda la sección siguiente.

```
p <- TRUE
q <- FALSE

p & q
p | q
!p
xor(p, q)
p == q
```

```
FALSE
TRUE
FALSE
TRUE
FALSE
```

Implicación

R no incluye un operador para la implicación lógica, pero eso no es un problema: la equivalencia $p \to q \equiv \neg p \vee q$ nos permite definirlo en una línea. De paso, esta es nuestra primera función.

```
implica <- function(p, q) {
  (!p) | q
}

implica(TRUE, TRUE)
implica(TRUE, FALSE)
implica(FALSE, TRUE)
implica(FALSE, FALSE)
```

```
TRUE
FALSE
TRUE
TRUE
```

Fíjate en los paréntesis alrededor de `!p`. Aquí no alteran el resultado, pero conviene adquirir el hábito, porque la precedencia de operadores en R guarda un par de sorpresas. Las dos que más vale la pena tener presentes son que `&` se evalúa antes que `|`, y que `!` se evalúa *después* que `==`.

```
p <- FALSE
q <- FALSE
r <- TRUE

# Como & tiene prioridad sobre |, estas dos expresiones no son la misma.
p & q | r      # R lo lee como (p & q) | r
p & (q | r)    # y esto es otra cosa
```

```
TRUE
FALSE
```

```
x <- 5
y <- 3

# Aquí la comparación se resuelve primero y la negación se aplica al final.
!x == y     # R lo lee como !(x == y)
(!x) == y   # en cambio aquí se niega x antes de comparar
```

```
TRUE
FALSE
```

Mientras trabajes sólo con valores lógicos la segunda trampa rara vez cambia el resultado, pero en cuanto haya números de por medio empieza a morder. La regla práctica es simple: si en una expresión conviven negaciones con comparaciones o con varios conectivos distintos, escribe los paréntesis aunque parezcan redundantes.

---

4. Tablas de verdad

Construir una tabla de verdad en R es sorprendentemente breve. La función `expand.grid()` genera todas las combinaciones posibles de los valores que le pases, es decir, exactamente el producto cartesiano que necesitamos.

```
tabla <- expand.grid(p = c(TRUE, FALSE),
                     q = c(TRUE, FALSE))
tabla
```

A data.frame: 4 × 2

p	q
<lgl>	<lgl>
TRUE	TRUE
FALSE	TRUE
TRUE	FALSE
FALSE	FALSE

Lo que acabas de obtener es un `data.frame`: una tabla con columnas nombradas, que es la estructura de datos más usada en R. Puedes acceder a cada columna con el signo `$`.

Como los operadores lógicos están vectorizados, agregar la columna de resultados para cualquier fórmula se reduce a escribir la fórmula tal cual.

```
tabla$conjuncion <- tabla$p & tabla$q
tabla$disyuncion <- tabla$p | tabla$q
tabla$implicacion <- implica(tabla$p, tabla$q)

tabla
```

A data.frame: 4 × 5

p	q	conjuncion	disyuncion	implicacion
<lgl>	<lgl>	<lgl>	<lgl>	<lgl>
TRUE	TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	TRUE	TRUE
TRUE	FALSE	FALSE	TRUE	FALSE
FALSE	FALSE	FALSE	FALSE	TRUE

Con tres variables el procedimiento es idéntico; sólo cambia el número de renglones, que pasa de cuatro a ocho.

```
tabla3 <- expand.grid(p = c(TRUE, FALSE),
                      q = c(TRUE, FALSE),
                      r = c(TRUE, FALSE))
```

```
tabla3$formula <- (tabla3$p & tabla3$q) | (!tabla3$r)
tabla3
```

A data.frame: 8 × 4

p	q	r	formula
<lgl>	<lgl>	<lgl>	<lgl>
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	TRUE	FALSE
TRUE	FALSE	TRUE	FALSE
FALSE	FALSE	TRUE	FALSE
TRUE	TRUE	FALSE	TRUE
FALSE	TRUE	FALSE	TRUE
TRUE	FALSE	FALSE	TRUE
FALSE	FALSE	FALSE	TRUE

---

5. Tautologías, contradicciones y equivalencias

Una vez que tenemos la columna de resultados, decidir si una fórmula es tautología o contradicción es cuestión de revisar esa columna completa. Las funciones `all()` y `any()` hacen justamente eso: la primera indica si **todos** los valores son verdaderos, la segunda si hay **al menos uno**.

```
es_tautologia <- function(columna) {
  all(columna)
}

es_contradiccion <- function(columna) {
  !any(columna)
}

# Ley del tercero excluido: p o no p.
tercero_excluido <- tabla$p | (!tabla$p)
es_tautologia(tercero_excluido)

# Principio de no contradicción: p y no p.
no_contradiccion <- tabla$p & (!tabla$p)
es_contradiccion(no_contradiccion)
```

```
TRUE
TRUE
```

Verificar una equivalencia

Dos fórmulas son lógicamente equivalentes si coinciden en todos los renglones de la tabla de verdad. Comprobemos con la primera ley de De Morgan, que afirma que $\neg(p \wedge q) \equiv \neg p \vee \neg q$.

Observa que calculo cada lado por separado antes de compararlos. Podría escribir todo en una sola línea, pero con negaciones de por medio el riesgo de precedencia que ya mencionamos es real, y el código queda además mucho más claro de leer.

```
lado_izquierdo <- !(tabla$p & tabla$q)
lado_derecho   <- (!tabla$p) | (!tabla$q)

# Comparamos renglón por renglón.
lado_izquierdo == lado_derecho

# Y resumimos el veredicto en un solo valor.
all(lado_izquierdo == lado_derecho)
```

```
TRUE · TRUE · TRUE · TRUE
TRUE
```

Este patrón —calcular ambos lados, compararlos, resumir con `all()`— es el que usarás para verificar cualquier equivalencia lógica de aquí en adelante.

6. De la lógica al agente

Un agente reactivo simple, en el sentido del Tema I, es aquel que elige su acción únicamente a partir de la percepción actual, sin memoria de lo ocurrido antes. Su comportamiento se describe mediante reglas de la forma *si condición entonces acción*, que no son otra cosa que implicaciones lógicas con una acción del lado derecho.

Vamos a modelar uno muy sencillo: un robot de limpieza que percibe tres cosas de su entorno.

```
# Una percepción es simplemente un conjunto de valores de verdad.
# Usamos una lista porque nos permite nombrar cada sensor.
percepcion_actual <- list(
  hay_obstaculo = FALSE,
  bateria_baja  = TRUE,
  hay_basura    = TRUE
)

percepcion_actual$bateria_baja
```

```
TRUE
```

Las reglas se implementan con una cadena de condicionales. El orden importa: la primera condición que se cumpla es la que determina la acción, de modo que estamos codificando también una **prioridad** entre las reglas.

```
decidir_accion <- function(p) {
  if (p$bateria_baja) {
    "ir a la base de carga"
  } else if (p$hay_obstaculo) {
    "girar"
  } else if (p$hay_basura) {
    "aspirar"
  } else {
    "avanzar"
  }
}

decidir_accion(percepcion_actual)
```

```
'ir a la base de carga'
```

En R, una función devuelve el valor de la última expresión que evalúa, así que no hace falta escribir `return()`. Aquí el `if` completo es esa última expresión, y su valor es la cadena correspondiente a la rama que se haya cumplido.

Ahora viene lo interesante. Podemos usar `expand.grid()` otra vez, pero no para una tabla de verdad sino para generar **todos los estados posibles del entorno**, y obtener así la tabla completa de comportamiento del agente.

```
escenarios <- expand.grid(
  hay_obstaculo = c(TRUE, FALSE),
  bateria_baja  = c(TRUE, FALSE),
  hay_basura    = c(TRUE, FALSE)
)

# mapply aplica la función a los tres vectores en paralelo, tomando un
# elemento de cada uno en cada paso. Es la forma idiomática de recorrer
# varias columnas a la vez sin escribir un ciclo.
escenarios$accion <- mapply(
  function(obstaculo, bateria, basura) {
    decidir_accion(list(hay_obstaculo = obstaculo,
                        bateria_baja  = bateria,
                        hay_basura    = basura))
  },
  escenarios$hay_obstaculo,
  escenarios$bateria_baja,
  escenarios$hay_basura
)

escenarios
```

A data.frame: 8 × 4

hay_obstaculo	bateria_baja	hay_basura	accion
<lgl>	<lgl>	<lgl>	<chr>
TRUE	TRUE	TRUE	ir a la base de carga
FALSE	TRUE	TRUE	ir a la base de carga
TRUE	FALSE	TRUE	girar
FALSE	FALSE	TRUE	aspirar
TRUE	TRUE	FALSE	ir a la base de carga
FALSE	TRUE	FALSE	ir a la base de carga
TRUE	FALSE	FALSE	girar
FALSE	FALSE	FALSE	avanzar

Esta tabla es, en sentido estricto, la especificación completa del agente: ocho percepciones posibles y la acción que corresponde a cada una. Vale la pena detenerse a mirarla, porque hace evidentes cosas que las reglas escritas en prosa esconden. Por ejemplo, cuando la batería está baja el agente ignora por completo la basura, incluso si la tiene enfrente.

Podemos contar cuántas veces aparece cada acción con `table()`, una función que usarás mucho a lo largo del semestre.

```
table(escenarios$accion)
```

```
             aspirar              avanzar                girar 
                   1                    1                    2 
ir a la base de carga 
                   4 
```

---

7. Un primer vistazo a los datos

Las prácticas siguientes trabajan con conjuntos de datos, de modo que conviene cerrar con lo básico para leerlos y explorarlos. R incluye varios conjuntos de ejemplo ya cargados; usaremos `iris`, que reúne medidas de flores de tres especies distintas y es el ejemplo clásico en problemas de clasificación.

```
# Las primeras filas, para hacernos una idea.
head(iris)

# Dimensiones: renglones y columnas.
dim(iris)
```

A data.frame: 6 × 5

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
	<dbl>	<dbl>	<dbl>	<dbl>	<fct>
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

150 · 5

Dos funciones que conviene usar siempre al recibir un conjunto de datos nuevo: `str()` describe la estructura y el tipo de cada columna, mientras que `summary()` da un resumen numérico.

```
str(iris)
```

```
'data.frame':   150 obs. of  5 variables:
 $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
```

```
 $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 $ Species     : Factor w/ 3 levels "setosa","versicolor",..: 1 1 1 1 1 1 1 1 1 1 ...
```

```
summary(iris)
```

```
  Sepal.Length    Sepal.Width     Petal.Length    Petal.Width
 Min.   :4.300   Min.   :2.000   Min.   :1.000   Min.   :0.100
 1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600   1st Qu.:0.300
 Median :5.800   Median :3.000   Median :4.350   Median :1.300
 Mean   :5.843   Mean   :3.057   Mean   :3.758   Mean   :1.199
 3rd Qu.:6.400   3rd Qu.:3.300   3rd Qu.:5.100   3rd Qu.:1.800
 Max.   :7.900   Max.   :4.400   Max.   :6.900   Max.   :2.500
       Species
 setosa    :50
 versicolor:50
 virginica :50
```

Nota que la columna `Species` no es texto sino un **factor**, que es el tipo con el que R representa variables categóricas. Internamente guarda un entero por cada categoría y una lista de etiquetas. Esto será importante en la Práctica 2, porque los clasificadores de R esperan que la variable de clase sea un factor.

```
class(iris$Species)
levels(iris$Species)
table(iris$Species)
```

```
'factor'
'setosa' · 'versicolor' · 'virginica'

    setosa versicolor  virginica
        50         50         50
```

Filtrar y resumir

El filtrado usa la misma indexación lógica de la sección 2, sólo que ahora sobre una tabla. La coma separa la condición sobre renglones de la selección de columnas.

```
# Renglones donde el pétalo mide más de 6 cm de largo.
grandes <- iris[iris$Petal.Length > 6, ]
grandes

# Promedio de largo de sépalo, calculado por especie.
tapply(iris$Sepal.Length, iris$Species, mean)
```

A data.frame: 9 × 5

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
	<dbl>	<dbl>	<dbl>	<dbl>	<fct>
106	7.6	3.0	6.6	2.1	virginica
108	7.3	2.9	6.3	1.8	virginica
110	7.2	3.6	6.1	2.5	virginica
118	7.7	3.8	6.7	2.2	virginica
119	7.7	2.6	6.9	2.3	virginica
123	7.7	2.8	6.7	2.0	virginica
131	7.4	2.8	6.1	1.9	virginica
132	7.9	3.8	6.4	2.0	virginica
136	7.7	3.0	6.1	2.3	virginica

```
setosa: 5.006 versicolor: 5.936 virginica: 6.588
```

Leer un archivo

Cuando los datos no vienen incluidos en R, se leen con `read.csv()`. Para practicarlo sin depender de una descarga, escribimos primero el conjunto a disco y lo volvemos a leer.

```
write.csv(iris, "iris_ejemplo.csv", row.names = FALSE)

datos <- read.csv("iris_ejemplo.csv")
str(datos)
```

```
'data.frame':   150 obs. of  5 variables:
 $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 $ Species     : chr  "setosa" "setosa" "setosa" "setosa" ...
```

Compara esta salida con la de `str(iris)` de hace unas celdas. La columna `Species` ya no es un factor sino texto, porque al escribir el archivo se perdió esa información. Es un detalle pequeño pero que causa errores con frecuencia; la conversión se hace con `as.factor()`.

```
datos$Species <- as.factor(datos$Species)
class(datos$Species)
```

```
'factor'
```

---

8. Ejercicios

Resuelve cada ejercicio en la celda de código que le corresponde. Escribe el código necesario para que la salida responda a lo que se pide; no basta con dejar la respuesta en un comentario.

Ejercicio 1

Construye la tabla de verdad de la fórmula $(p \vee q) \wedge \neg r$ para las tres variables p, q y r. La tabla debe tener ocho renglones y mostrar tanto las variables como el resultado.

```
# Ejercicio 1
```

```
# Tabla de verdad para (p | q) & !r
tabla_e1 <- expand.grid(p = c(TRUE, FALSE),
                        q = c(TRUE, FALSE),
                        r = c(TRUE, FALSE))

tabla_e1$resultado <- (tabla_e1$p | tabla_e1$q) & (!tabla_e1$r)
tabla_e1
```

A data.frame: 8 × 4

p	q	r	resultado
<lgl>	<lgl>	<lgl>	<lgl>
TRUE	TRUE	TRUE	FALSE
FALSE	TRUE	TRUE	FALSE
TRUE	FALSE	TRUE	FALSE
FALSE	FALSE	TRUE	FALSE
TRUE	TRUE	FALSE	TRUE
FALSE	TRUE	FALSE	TRUE
TRUE	FALSE	FALSE	TRUE
FALSE	FALSE	FALSE	FALSE

Ejercicio 2

Verifica la **segunda ley de De Morgan**, es decir $\neg(p \vee q) \equiv \neg p \wedge \neg q$. Calcula cada lado por separado, compáralos renglón por renglón y resume el resultado en un solo valor lógico.

```
# Ejercicio 2
```

```
# Segunda ley de De Morgan: !(p | q) == !p & !q
tabla_e2 <- expand.grid(p = c(TRUE, FALSE),
                        q = c(TRUE, FALSE))

lado_izq_e2 <- !(tabla_e2$p | tabla_e2$q)
lado_der_e2 <- (!tabla_e2$p) & (!tabla_e2$q)

# Comparación renglón por renglón
lado_izq_e2 == lado_der_e2

# Veredicto final
all(lado_izq_e2 == lado_der_e2)
```

```
TRUE · TRUE · TRUE · TRUE
TRUE
```

Ejercicio 3

Usando la función `implica()` definida en la sección 3, comprueba que el **silogismo hipotético** es una tautología:

$$((p \to q) \land (q \to r)) \to (p \to r)$$

Necesitarás una tabla de tres variables.

```
# Ejercicio 3
```

```
# Silogismo hipotético: ((p -> q) & (q -> r)) -> (p -> r)
tabla_e3 <- expand.grid(p = c(TRUE, FALSE),
                        q = c(TRUE, FALSE),
                        r = c(TRUE, FALSE))
tabla_e3
```

A data.frame: 8 × 3

p	q	r
<lgl>	<lgl>	<lgl>
TRUE	TRUE	TRUE
FALSE	TRUE	TRUE
TRUE	FALSE	TRUE
FALSE	FALSE	TRUE
TRUE	TRUE	FALSE
FALSE	TRUE	FALSE
TRUE	FALSE	FALSE
FALSE	FALSE	FALSE

```
premisa_1 <- implica(tabla_e3$p, tabla_e3$q)
premisa_2 <- implica(tabla_e3$q, tabla_e3$r)
antecedente <- premisa_1 & premisa_2
consecuente <- implica(tabla_e3$p, tabla_e3$r)

tabla_e3$resultado <- implica(antecedente, consecuente)
tabla_e3

# Comprobación de que es una tautología
es_tautologia(tabla_e3$resultado)
```

A data.frame: 8 × 4

p	q	r	resultado
<lgl>	<lgl>	<lgl>	<lgl>
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	TRUE	TRUE
TRUE	FALSE	TRUE	TRUE
FALSE	FALSE	TRUE	TRUE
TRUE	TRUE	FALSE	TRUE
FALSE	TRUE	FALSE	TRUE
TRUE	FALSE	FALSE	TRUE
FALSE	FALSE	FALSE	TRUE

TRUE

Ejercicio 4

Modifica el agente de la sección 6 agregando un cuarto sensor, `es_de_noche`, con la siguiente regla: si es de noche, el agente debe detenerse sin importar lo demás, salvo que la batería esté baja, en cuyo caso conserva la prioridad de ir a cargarse.

Escribe la nueva función y genera la tabla de comportamiento completa. Ahora serán dieciséis escenarios.

```
# Ejercicio 4
```

```
# Función del agente reactivo con 4 sensores
decidir_accion_e4 <- function(p) {
  if (p$bateria_baja) {
    "ir a la base de carga"
  } else if (p$es_de_noche) {
    "detenerse"
  } else if (p$hay_obstaculo) {
    "girar"
  } else if (p$hay_basura) {
    "aspirar"
  } else {
    "avanzar"
  }
}

# Generar los 16 escenarios posibles
escenarios_e4 <- expand.grid(
  hay_obstaculo = c(TRUE, FALSE),
  bateria_baja  = c(TRUE, FALSE),
  hay_basura    = c(TRUE, FALSE),
  es_de_noche   = c(TRUE, FALSE)
)

escenarios_e4$accion <- mapply(
  function(obstaculo, bateria, basura, noche) {
    decidir_accion_e4(list(
      hay_obstaculo = obstaculo,
      bateria_baja  = bateria,
      hay_basura    = basura,
      es_de_noche   = noche
    ))
  },
  escenarios_e4$hay_obstaculo,
  escenarios_e4$bateria_baja,
  escenarios_e4$hay_basura,
  escenarios_e4$es_de_noche
)

escenarios_e4
```

A data.frame: 16 × 5

hay_obstaculo	bateria_baja	hay_basura	es_de_noche	accion
<lgl>	<lgl>	<lgl>	<lgl>	<chr>
TRUE	TRUE	TRUE	TRUE	ir a la base de carga
FALSE	TRUE	TRUE	TRUE	ir a la base de carga
TRUE	FALSE	TRUE	TRUE	detenerse
FALSE	FALSE	TRUE	TRUE	detenerse
TRUE	TRUE	FALSE	TRUE	ir a la base de carga
FALSE	TRUE	FALSE	TRUE	ir a la base de carga
TRUE	FALSE	FALSE	TRUE	detenerse
FALSE	FALSE	FALSE	TRUE	detenerse
TRUE	TRUE	TRUE	FALSE	ir a la base de carga
FALSE	TRUE	TRUE	FALSE	ir a la base de carga
TRUE	FALSE	TRUE	FALSE	girar
FALSE	FALSE	TRUE	FALSE	aspirar
TRUE	TRUE	FALSE	FALSE	ir a la base de carga
FALSE	TRUE	FALSE	FALSE	ir a la base de carga
TRUE	FALSE	FALSE	FALSE	girar
FALSE	FALSE	FALSE	FALSE	avanzar

Ejercicio 5

Sobre el conjunto `iris`:

1. Obtén el promedio de `Petal.Width` para cada especie.
2. Cuenta cuántas flores tienen `Sepal.Width` mayor a 3.5.
3. Muestra únicamente los renglones de la especie *virginica* cuyo `Petal.Length` sea menor al promedio general de esa columna.

```
# Ejercicio 5
```

```
# 1. Promedio de Petal.Width por especie
tapply(iris$Petal.Width, iris$Species, mean)

# 2. Conteo de flores con Sepal.Width > 3.5
sum(iris$Sepal.Width > 3.5)

# 3. Renglones de virginica con Petal.Length menor al promedio general
promedio_petal_length <- mean(iris$Petal.Length)
iris[iris$Species == "virginica" & iris$Petal.Length < promedio_petal_length, ]
```

```
setosa:     0.246 versicolor:     1.326 virginica:     2.026
19
```

A data.frame: 0 × 5

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
<dbl>	<dbl>	<dbl>	<dbl>	<fct>

Ejercicio 6

Traduce el siguiente enunciado a una expresión lógica en R y construye su tabla de verdad:

> El sistema emite una alerta si el sensor está activo y hay movimiento, o bien si el sensor está activo y la puerta no está cerrada.

Después, determina si esa expresión es equivalente a `sensor_activo & (movimiento | !puerta_cerrada)`.

```
# Ejercicio 6
```

```
# Tabla de verdad del enunciado
tabla_e6 <- expand.grid(
  sensor_activo = c(TRUE, FALSE),
  movimiento    = c(TRUE, FALSE),
  puerta_cerrada = c(TRUE, FALSE)
)

# Traducción: (sensor & movimiento) | (sensor & !puerta_cerrada)
expresion_1 <- (tabla_e6$sensor_activo & tabla_e6$movimiento) |
               (tabla_e6$sensor_activo & (!tabla_e6$puerta_cerrada))

tabla_e6$alerta <- expresion_1
tabla_e6

# Verificación de equivalencia con sensor_activo & (movimiento | !puerta_cerrada)
expresion_2 <- tabla_e6$sensor_activo & (tabla_e6$movimiento | (!tabla_e6$puerta_cerrada))

all(expresion_1 == expresion_2)
```

A data.frame: 8 × 4

sensor_activo	movimiento	puerta_cerrada	alerta
<lgl>	<lgl>	<lgl>	<lgl>
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	TRUE	FALSE
TRUE	FALSE	TRUE	FALSE
FALSE	FALSE	TRUE	FALSE
TRUE	TRUE	FALSE	TRUE
FALSE	TRUE	FALSE	FALSE
TRUE	FALSE	FALSE	TRUE
FALSE	FALSE	FALSE	FALSE

TRUE

---

9. Reflexión

Responde en esta misma celda, con tus palabras y en un párrafo por pregunta. Se valora la claridad del razonamiento, no la extensión.

1. En la sección 6 obtuvimos una tabla con las ocho percepciones posibles y la acción correspondiente. Un agente reactivo simple puede, en principio, describirse completamente con una tabla así. ¿Qué ocurre con el tamaño de esa tabla conforme agregamos sensores, y qué consecuencia tiene eso para construir agentes que operen en entornos reales?

Tu respuesta: Conforme sumamos sensores, el tamaño de la tabla se duplica con cada uno (crece de forma exponencial como 2^n). Al principio, con tres o cuatro sensores, tener 8 o 16 combinaciones es muy fácil de manejar, pero en un entorno real donde un agente recibe decenas de variables, la tabla se vuelve tan gigantesca que resultaría imposible guardarla o consultarla en memoria. Por eso, en la práctica no conviene crear listas infinitas con todas las combinaciones posibles, sino diseñar reglas de decisión o modelos que determinen qué hacer en el momento sin tener que registrar cada caso por adelantado.

2. Todas las fórmulas de esta práctica se evaluaron construyendo la tabla de verdad completa y revisándola. Ese procedimiento siempre funciona y siempre da la respuesta correcta. ¿Por qué, entonces, no es una estrategia satisfactoria para determinar si una fórmula con muchas variables es tautología? Relaciona tu respuesta con lo visto sobre búsqueda y espacios de estados.

Tu respuesta: Construir la tabla de verdad completa siempre da la respuesta correcta, pero no es una estrategia práctica cuando hay muchas variables porque el espacio de estados crece demasiado rápido. Si lo vemos como un problema de búsqueda, revisar la tabla renglón por renglón equivale a explorar cada camino a ciegas en lugar de usar atajos, reglas de inferencia o métodos guiados que permitan demostrar la verdad de la fórmula mucho más rápido y sin tener que recorrer todo el árbol de combinaciones.

Criterios de evaluación

Aspecto	Puntos
Ejercicios 1 a 3 (lógica proposicional)	30
Ejercicio 4 (agente reactivo)	20
Ejercicios 5 y 6	20
Reflexión	20
Cuaderno ejecutado de principio a fin, sin errores y con salidas visibles	10

Aspecto	Puntos
Total	**100**

Un cuaderno que no corra completo tras reiniciar el entorno pierde los diez puntos correspondientes y los ejercicios cuyas celdas fallen.

Pregunta 1

El tamaño de la tabla crece de manera exponencial como 2^n, donde n representa la cantidad de sensores o variables binarias de percepción. En entornos reales con decenas o cientos de estímulos, este crecimiento vuelve intratable la representación explícita del comportamiento del agente, pues requeriría un espacio de memoria y una capacidad computacional desmedidos para generar y almacenar todas las combinaciones posibles. Por esta razón, en lugar de tablas exhaustivas, los agentes prácticos emplean estructuras compactas como árboles de decisión, redes lógicas, bases de conocimiento o algoritmos de aprendizaje que generalizan patrones a partir de la entrada sin evaluar explícitamente todo el espacio de estados.

Pregunta 2

Construir la tabla de verdad completa equivale a realizar una búsqueda no informada a través de un espacio de estados cuyo tamaño es 2^n respecto al número de variables proposicionales. Aunque garantiza exhaustividad y corrección, la complejidad temporal en el peor caso es de orden $O(2^n)$, lo que ocasiona una explosión combinatoria al aumentar la cantidad de variables. En lógica proposicional, determinar si una fórmula es satisfacer o tautología corresponde al problema SAT, por lo que resulta mucho más eficiente utilizar algoritmos de inferencia formal, resolución, algoritmos tipo DPLL o heurísticas de búsqueda local que navegan el espacio de estados sin explorar sistemáticamente cada combinación.

Conclusiones finales del estudiante.